

Architecture Details: Bilingual Road-side Text Detection and Recognition

1 System overview

The system reads Hindi and English text in dash-camera video in five stages: a text detector finds word regions, a rectification step straightens each word, a recogniser reads it, a lexicon corrects low-confidence readings, and a tracker combines the readings of the same sign across frames. The two neural networks are trained from random initialisation; no pretrained weights are used.

Table 1: Pipeline stages.

Stage	Input	Output
Resize and pad	video frame	longer side scaled to 1600–2000 px, padded to a multiple of 32
Text detector	padded frame	kernel and region probability maps at 1/2 resolution
Post-processing	probability maps	word boxes as four-point quadrilaterals
Rectification	frame and quadrilateral	upright word crop (8% padding, at most 96 px high)
Recogniser	word crop	text and confidence score
Lexicon	text and score	corrected text (only if score < 0.9)
Tracker and voting	readings of all frames	one final reading per sign

2 Text detector

The detector is a fully convolutional segmentation network in the style of Differentiable Binarization (DB). Instead of regressing boxes, it predicts, for every pixel, whether the pixel belongs to a word. Two maps are predicted: a *kernel map*, which covers a shrunk version of each word, and a *region map*, which covers the full word and is used as auxiliary supervision. Shrinking keeps neighbouring words separate; at inference the detected kernels are expanded back to full word size.

2.1 Layers

Each residual block computes

$$y = \text{ReLU}(\text{BN}(W_2 * \text{ReLU}(\text{BN}(W_1 * x))) + S(x)), \quad (1)$$

where W_1, W_2 are 3×3 convolutions and S is the identity, or a 1×1 convolution with batch normalisation when the stride or the number of channels changes. All convolutions are bias-free and followed by batch normalisation, except the final 1×1 output convolution.

Table 2: Detector layers. Output sizes are for a 640×640 training crop; at inference any size divisible by 32 is accepted. Parameter counts include the non-trainable batch-normalisation statistics.

Stage	Operation	Output	Parameters
Input	$x/127.5 - 1$	$640 \times 640 \times 3$	0
Stem	Conv $3 \times 3/2$ (32), Conv 3×3 (32)	$320 \times 320 \times 32$	10,336
C2	2 residual blocks, 48 channels, first with stride 2	$160 \times 160 \times 48$	78,528
C3	2 residual blocks, 96 channels	$80 \times 80 \times 96$	296,832
C4	3 residual blocks, 160 channels	$40 \times 40 \times 160$	1,310,080
C5	2 residual blocks, 256 channels	$20 \times 20 \times 256$	2,184,192
FPN lateral	Conv 1×1 to 96 channels on C2–C5; top-down $2 \times$ upsampling and addition	$\{160, 80, 40, 20\}^2 \times 96$	55,296
FPN fusion	Conv 3×3 to 48 channels per level, upsample to $1/4$, concatenate	$160 \times 160 \times 192$	166,656
Head	Conv 3×3 (96), bilinear $2 \times$ upsampling, Conv 3×3 (48), Conv 1×1 (2)	$320 \times 320 \times 2$	208,034
Total			4,309,954

2.2 Training targets

For a word polygon with area A and perimeter L , the kernel is obtained by moving every edge inwards by

$$D = \frac{A(1 - r^2)}{L}, \quad r = 0.6. \quad (2)$$

The region target is the full polygon. Words marked illegible and words less than 6 px high are excluded from the loss through a weight map w that is zero over them and one elsewhere. All targets are drawn at $1/2$ resolution.

2.3 Loss

For a map with predicted probability p , ground truth g and weight w , the balanced binary cross-entropy keeps all positive pixels and only the hardest negative pixels, at most three times the number of positives (and at least 256):

$$\mathcal{L}_{\text{bce}} = \frac{\sum_{i \in P} \ell_i + \sum_{i \in N_k} \ell_i}{|P| + k}, \quad k = \min(|N|, \max(3|P|, 256)), \quad (3)$$

where ℓ_i is the per-pixel cross-entropy, P the positive pixels, N the negative pixels, and N_k the k negatives with the largest loss. The Dice loss measures overlap over the whole map:

$$\mathcal{L}_{\text{dice}} = 1 - \frac{2 \sum_i p_i g_i w_i}{\sum_i p_i w_i + \sum_i g_i w_i}. \quad (4)$$

The detector loss combines both terms on the kernel map and, with weight 0.5, on the region map:

$$\mathcal{L}_{\text{det}} = \mathcal{L}_{\text{bce}}^{\text{kernel}} + \mathcal{L}_{\text{dice}}^{\text{kernel}} + 0.5 (\mathcal{L}_{\text{bce}}^{\text{region}} + \mathcal{L}_{\text{dice}}^{\text{region}}). \quad (5)$$

2.4 Post-processing

The kernel map is binarised with threshold t_{bin} , and each connected component is enclosed by its minimum-area rotated rectangle. A box is kept if the mean kernel probability inside the component

is at least t_{box} . The rectangle, of area A' and perimeter L' , is then expanded by

$$D' = \frac{A' u}{L'}, \quad (6)$$

where u is the unclip ratio, and mapped back to frame coordinates. At most 300 boxes are kept per frame, and boxes covering more than 60% of the frame are discarded. The values of t_{bin} , t_{box} and u are chosen by grid search on a held-out part of the training split (never on the test split); the calibrated values were $t_{\text{bin}} = 0.45$, $t_{\text{box}} = 0.75$ and $u = 0.9$.

3 Recogniser

The recogniser is a convolutional recurrent neural network (CRNN) trained with connectionist temporal classification (CTC). Convolutions reduce the word crop to a left-to-right sequence of feature vectors, two bidirectional LSTM layers read this sequence, and a linear layer gives a probability for every character at every time step.

3.1 Input

Each crop is converted to one grey channel: the channel (grey, blue, green, red or saturation) with the strongest Otsu separation is chosen, the polarity is normalised so that the background is dark, and the contrast is stretched between the 2nd and 98th percentiles. The crop is then scaled without distortion and left-aligned in a 48×320 canvas. A height of 48 px, rather than the usual 32 px, keeps the Devanagari vowel signs above and below the headline.

3.2 Layers

Table 3: Recogniser layers. All convolutions are 3×3 , bias-free and followed by batch normalisation and ReLU, unless stated otherwise.

Operation	Output	Parameters
Input (grey, scaled to $[0, 1]$)	$48 \times 320 \times 1$	0
Conv (32), max-pool 2×2	$24 \times 160 \times 32$	416
Conv (64), max-pool 2×2	$12 \times 80 \times 64$	18,688
Conv (128)	$12 \times 80 \times 128$	74,240
Conv (128), max-pool 2×1	$6 \times 80 \times 128$	147,968
Conv (256)	$6 \times 80 \times 256$	295,936
Conv (256), max-pool 2×1	$3 \times 80 \times 256$	590,848
Conv 3×1 valid (320), collapses the height	$1 \times 80 \times 320$	247,040
Reshape, dropout 0.2	80×320	0
Bidirectional LSTM (256 per direction)	80×512	1,181,696
Bidirectional LSTM (256 per direction)	80×512	1,574,912
Dropout 0.2, dense (224)	80×224	114,912
Total		4,246,656

The 224 output classes are the CTC blank and 223 characters: 95 printable Latin characters (letters, digits, punctuation and space) and the 128 assigned code points of the Unicode Devanagari block. One model reads both scripts; no language switch is needed.

3.3 Loss and decoding

For an input x with label y , CTC sums over all frame-level paths π that collapse to y :

$$\mathcal{L}_{\text{ctc}} = -\log \sum_{\pi \in \mathcal{B}^{-1}(y)} \prod_{t=1}^T p(\pi_t | x), \quad T = 80, \quad (7)$$

where $\mathcal{B}$ removes repeated labels and then blanks. The 80 time steps allow words of up to 32 characters, since CTC needs at least $2n - 1$ steps for n characters with repeats. Decoding is greedy: the most likely class is taken at each step, repeats are collapsed and blanks removed. The confidence score is the mean of the peak probability of each emitted character.

Devanagari labels are stored in visual order: the vowel sign i , which is drawn before its consonant cluster but written after it in Unicode, is moved before the cluster for training and moved back after decoding. This matches the left-to-right order in which the network sees the glyphs.

4 Lexicon correction and video voting

Lexicon. A word list is built from the training-split transcriptions and road-sign vocabularies (15,419 entries after phase 2). A reading is replaced by its nearest lexicon word only if its confidence is below 0.9, it has at least three characters, it contains no digits, it is not already in the lexicon, and its Levenshtein distance to that word is at most 1 (up to 5 characters), 2 (up to 10) or 3 (longer words).

Tracker. Detections in consecutive frames are matched greedily by axis-aligned intersection over union (threshold 0.3). A track stays alive for up to $\text{round}(f/3)$ frames without a match, where f is the frame rate. Each track accumulates votes for the texts read in its frames, with each vote weighted by the reading’s confidence score. The track’s final text is the one with the largest total weight. Per-frame readings are kept unchanged; voting only adds one final answer per sign.

5 Training setup

After training, the batch-normalisation statistics of both networks are recomputed as the exact average over 40 training batches (precise BN). Without this step, short trainings keep part of the initial statistics and the network behaves differently in inference mode; in a memorisation check, the recogniser read 0 of 16 words before the refresh and 16 of 16 after it. Batch sizes were chosen from GPU memory (16 GB T4) and common practice, and were not tuned.

Table 4: Training settings. Both networks use Adam with gradient-norm clipping at 5, halve the learning rate when the validation loss stops improving, stop early, and keep the weights with the lowest validation loss.

Setting	Detector	Recogniser
Input	640×640 random crops	48×320 grey crops
Batch size	8	64
Phase 1 data	about 4,000 synthetic road scenes	60,000 synthetic sign-style word crops
Phase 1 schedule	20 epochs $\times$ 400 steps, lr 10^{-3}	20 epochs $\times$ 800 steps, lr 10^{-3}
Phase 2 data	BSTD train images 75%, synthetic 25%	real BSTD word crops 60%, synthetic 40%
Phase 2 schedule	30 epochs $\times$ 500 steps, lr 5×10^{-4} , from phase 1	20 epochs $\times$ 800 steps, lr 5×10^{-4} , from phase 1
Fine-tuning	–	20,000 real Hindi word crops 80%, synthetic 20%; 15 epochs $\times$ 600 steps, lr 3×10^{-4}
Plateau schedule	factor 0.5, patience 3, minimum 10^{-5}	factor 0.5, patience 2, minimum 10^{-5}
Early stopping	patience 8	patience 6
Precision	mixed float16 on GPUs with compute capability ≥ 7.0	float32
Augmentation	scale $\times 0.5$ –1.8 around the inference scale, text-biased cropping, rotation up to $\pm 10^\circ$, brightness, colour cast, saturation, shading, blur, motion blur, down-scaling, noise, JPEG	detector-like jitter of the word box, 0–15% padding, perspective, shading, blur, down-scaling, noise, JPEG